

Retrieval, Vectors, Embeddings, & Vector Databases

Saraansh Wadkar

1 The Retrieval Problem

Suppose we have a collection of documents and a user asks a question. The system needs to find the parts of those documents that are relevant to the query. The documents might be product descriptions, support tickets, legal contracts, code files, research papers, or internal company notes. The same basic problem keeps showing up: given a query, return the text that is most likely to help.

1.1 Raw Keyword Search

The simplest version is keyword search. If the query is “Toyota RAV4,” we can look for catalog entries that contain the words “Toyota” and “RAV4.” This works better than it sounds. Exact terms often matter a lot. Model names, trims, years, drivetrain labels, engine codes, and VINs usually need literal matching.

But raw keyword search is brittle. Suppose a user asks for:

“vehicle good for off-road adventures”

The best result might say “compact SUV with all-wheel drive” or “pickup truck built for rough terrain.” Those documents are relevant, but the exact query words may not appear. A literal keyword search can miss them for a boring reason: the words do not overlap enough.

1.2 TF-IDF: Making Word Counts Less Naive

One natural improvement is to score a document by counting how often the query words appear. If the query is “electric sedan,” a listing that mentions “electric” and “sedan” several times probably deserves more attention than a listing that mentions each word once in passing. This is the term frequency part: how often does this term appear here?

But raw counts have an obvious problem. Some words are everywhere. Words like “the,” “is,” and “good” should not carry much weight. A rare term like RAV4, AWD, “hybrid,” or “hatchback” should matter much more because it narrows the search quickly.

That gives us the intuition behind TF-IDF. Term frequency asks how much a term appears in this document. Inverse document frequency asks how rare that term is across the whole collection. A useful search term is often one that appears in this document but does not appear everywhere.

TF-IDF is still lexical search, though. It can rank overlapping words more carefully, but it still needs overlap. If the query says “vehicle” and the document says “SUV,” TF-IDF does not know those ideas are related.

1.3 BM25: A Stronger Lexical Baseline

TF-IDF gets the basic rare-word intuition right, but practical ranking has a few more wrinkles. After TF-IDF, the next question is not just which words matter. It is also how repeated matches should change the score.

Repetition helps, but only up to a point. The first few mentions of a term like “electric” are strong evidence that a listing is really about an electric car. Later mentions still help, but each extra mention tells us less than the previous one. In other words, repetition should have diminishing returns. A document that mentions “electric” five times is probably more relevant than one that mentions it once. A document that repeats “electric” 50 times is not 50 times better.

Document length causes another problem. Long documents have more chances to contain any query term, so they can look relevant just because they have more words. A long dealership page or family-car roundup naturally has more opportunities to mention “electric,” “SUV,” or “AWD” than a short listing does. But that does not mean it is the better match. A short listing that says “electric sedan with long range” should probably rank above a long overview that mentions “electric sedan” once among 20 different vehicles.

BM25 is a practical way to rank lexical matches by combining three intuitions: some terms matter more than others, repeated matches should help with diminishing returns, and long documents need a correction so they do not win by default. It is still based on words, but it is a strong baseline for exact terms: model names, trims, VINs, drivetrain labels, engine types, and rare phrases. When exact wording matters, BM25 is often the right tool.

1.4 Where Lexical Search Still Breaks

TF-IDF and BM25 are useful, but they still mostly match words rather than meaning. They struggle with synonyms, paraphrases, and vague natural-language queries. “Car” and “vehicle” may refer to the same thing. “Good for rough terrain” and “built for off-road use” are close in meaning without being the same sentence. A document that says “compact SUV with all-wheel drive” may still be a good answer to a query about “off-road adventures,” even if the words barely line up.

This is the gap semantic search is trying to fill. Instead of comparing text only by word overlap, semantic search maps both the documents and the query into vectors. Those vectors are arranged so that related meanings tend to land near each other. The search problem changes from “which documents contain these words?” to “which stored meanings are closest to this query?”

2 Semantic Search

Semantic search works by turning both queries and documents into vectors that preserve meaning geometrically. If related ideas land near each other, then retrieval can work from semantic closeness instead of relying only on overlapping words.

2.1 What Is a Vector?

In the context of embeddings and vector databases, a vector is an n -dimensional ordered list of numbers:

$$\mathbf{v} = [v_1, v_2, \dots, v_n].$$

In practice, each coordinate is usually stored as a signed 32-bit floating-point value, so a single vector is often represented as an n -dimensional array of 4-byte floats.

An embedding vector is useful because it encodes semantic meaning into geometry. Rather than storing meaning in words or symbols directly, the model maps text, images, or other data into a point in a high-dimensional space. Items with similar meaning are placed closer together, while items with different meanings are placed farther apart.

For example, two sentences about databases may produce vectors that point in similar directions, even if they do not share the exact same words. This is what makes vector search possible: we compare the geometry of embeddings instead of relying only on exact keyword matches.

2.2 How Embeddings Are Created

The key question is how a piece of text becomes a useful vector at all. An embedding model first breaks text into smaller units called tokens. Those tokens are then processed by a neural network that has been trained on very large amounts of data. During training, the model learns internal patterns about which words, phrases, and concepts tend to appear in similar contexts.

By the time the model is used for embeddings, it has learned to map text into a high-dimensional latent space. Each coordinate in the output vector is a learned scalar value. The model was not told ahead of time that dimension 17 should mean “animal” or that dimension 243 should mean “database.” Those labels are not built in. Still, the numbers are not random. Training pushes the model to arrange them so that useful patterns show up in the geometry of the space.

This is why embeddings are deeper than just lists of floats. A coordinate can carry semantic signal, but that signal is usually not cleanly labeled or easy to isolate. One direction in the space might roughly line up with something like “more technical,” “more emotional,” “more physical object,” or “more finance-related.” That does not mean there is a neat technicalness dimension that we can read directly. It means the model has learned directions and combinations of coordinates that tend to matter for the data it saw during training.

The sign and size of a scalar matter too, but only inside the learned space. A positive value points one way along that coordinate or direction. A negative value points the opposite way. A larger absolute value means that coordinate contributes more strongly to the vector’s position. However, a single coordinate rarely explains the meaning by itself. The meaning

is usually spread across many dimensions, and the full vector matters as a whole. Two embeddings end up close because many of these learned signals line up together. That is why cosine similarity compares the direction of the whole vector rather than reading one number at a time.

For a deeper intuition, 3Blue1Brown has a short video on how directions or dimensions in these spaces can pick up semantic meaning in this 3Blue1Brown video.

For example, suppose we embed the sentence “the cat sat on the mat.” The model first tokenizes it into pieces such as “the”, “cat”, “sat”, “on”, “the”, and “mat”. Those tokens are passed through many layers of the network, which combine all of those learned signals into a single output vector such as

$$\mathbf{e}_{\text{cat}} = [0.12, -0.44, 0.91, \dots, 0.08].$$

The exact numbers are not human-interpretable on their own. What matters is that another sentence like “a kitten is resting on a rug” will likely produce a vector in a nearby region of the space, because the model has learned that “cat” and “kitten” are related concepts and that both sentences describe similar situations.

As a result, text with similar meaning tends to produce vectors that end up near each other, even when the wording changes. The embedding works because the model has compressed many layers of statistical and semantic structure into a numerical representation that preserves useful similarity relationships.

2.3 Semantic Search Example

Suppose you are shopping for a car and want something good for off-road adventures. You have a small catalog of cars, each with a short description. At first this sounds like a simple search problem, but there is an immediate issue: none of the descriptions may literally contain the phrase “off-road adventures.” A plain keyword search or **Ctrl+F** style lookup would therefore miss relevant cars even if some of them are actually a good fit.

Here is the dataset:

1. “Tesla Model 3 is an electric sedan designed for daily commuting with great efficiency”
2. “Ford F-150 is a powerful pickup truck built for hauling and towing heavy loads”
3. “Toyota RAV4 is a compact SUV with good fuel economy and family-friendly features”
4. “Porsche 911 is a high-performance sports car built for speed and handling”
5. “Honda Civic is a reliable compact car ideal for commuting”
6. “Chevy Bolt is an affordable electric hatchback for city driving”

The Ford F-150 and Toyota RAV4 might both be relevant to an “off-road adventures” style query, but neither description uses those exact words. So how do we search by meaning instead of by exact text?

The key idea is to embed each description into a vector. Conceptually, each record might look like:

```
{
  car_name: "Tesla Model 3",
  description: "Electric sedan for efficient daily commuting",
  vector: [...]
}
```

The important point is that the **vector** is produced from the **description**. Once all the descriptions are embedded, we do the same thing to the user’s query:

“vehicle good for off-road adventures”

In this situation, the Toyota RAV4 and Ford F-150 might rank higher than the Tesla Model 3 or Honda Civic, even though none of the descriptions contain the exact phrase “off-road adventures.” The system works because the embedding model has placed related meanings in nearby parts of the vector space. This is the core promise of semantic search: retrieve by meaning, not just by literal keyword overlap.

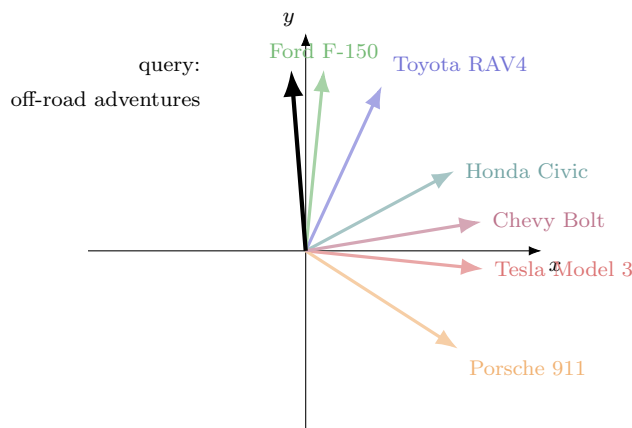


Figure 1: A 2D picture of semantic similarity. Real embeddings are high-dimensional, but the same directional intuition applies.

This picture is only 2D, while real embeddings live in much higher dimensions, but it captures the basic idea. Cars that are close together have similar descriptions. The Tesla Model 3, Honda Civic, and Chevy Bolt cluster in a more commuting- and city-driving-oriented direction. The Toyota RAV4 and Ford F-150 sit much closer to each other because their descriptions both emphasize utility. The query vector for “off-road adventures” also points in that same general direction, which is why those vehicles would be better matches.

At that point, the natural question becomes: how can we mathematically find which vectors are most similar?

2.3.1 Cosine Similarity

Once we have a query vector and a set of car vectors, we need a mathematical way to ask: which vectors point in the most similar direction? One of the most common answers is cosine

similarity. Given vectors $\mathbf{u}$ and $\mathbf{v}$, their cosine similarity is

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

Intuitively, this is about angle. If two vectors point in almost the same direction, then the angle between them is close to 0° , and $\cos(0^\circ) = 1$. That means the vectors are highly similar. If the angle is around 90° , then $\cos(90^\circ) = 0$, which means they are effectively unrelated. If they point in opposite directions, the angle approaches 180° and the cosine becomes negative.

We can make that concrete by isolating the query vector and comparing it against a few individual cars:

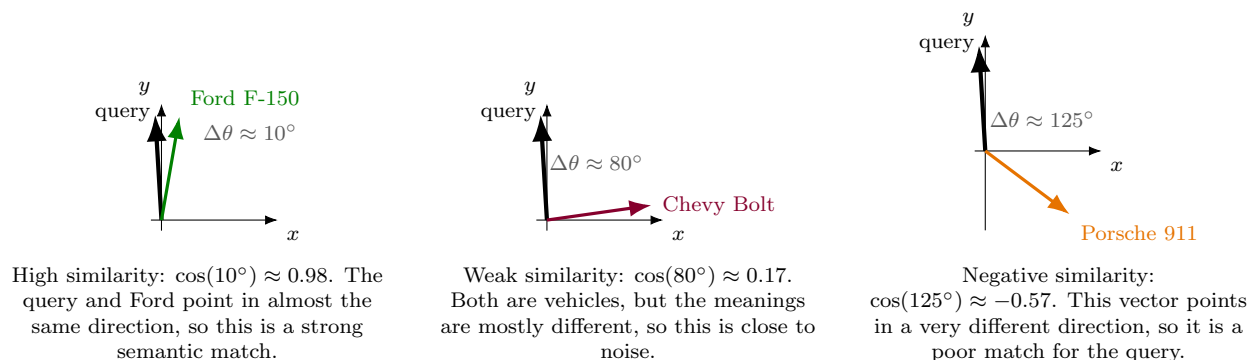


Figure 2: Cosine similarity compares the angle between the query vector and each car vector. Small angles mean high similarity; large angles mean weak or even negative similarity.

In the car-shopping example, we would compute the cosine similarity between the query vector and each car description vector, then sort the results from highest to lowest score. If the query vector points in a similar direction to the Toyota RAV4 vector, that suggests their meanings are related even though the query says “off-road adventures” and the description says “compact SUV.” This is exactly why cosine similarity is useful: it lets us retrieve semantically related results without needing the same literal words to appear in both strings.

In practice, cosine similarity scores are often interpreted as rough relevance signals. A score around 0.9 usually indicates a very strong match. Around 0.7 is often still meaningfully related, but less precise. Around 0.4 is more borderline: there may be some overlap in topic or context, but the match is weak. Around 0.2 is often close to noise. These are heuristics rather than hard rules, since the exact thresholds depend on the embedding model and the dataset.

The same idea is not limited to language. Vector representations can also capture similarity for numbers and other structured features. If two data points have similar numerical patterns, their vectors may still end up close together, allowing the database to retrieve values that are numerically or structurally related rather than only textually related.

2.4 Piecing it together

At this point, we have two ingredients: a way to turn data into vectors and a way to compare those vectors. The natural next question is what kind of structure would let us use those

ingredients in a real system. Having individual vectors is not enough on its own. We need some organized way to store them alongside the data they came from so that a query can return something useful to a user rather than just a list of floating-point numbers.

The simplest useful structure is a collection of records where each record contains the original content, its embedding vector, and whatever extra metadata we may want to keep around. Conceptually, a single entry might look like this:

```
{
  id: 42,
  content: "Toyota RAV4 is a compact SUV with good fuel economy",
  embedding: [...],
  metadata: {
    brand: "Toyota",
    type: "SUV"
  }
}
```

Once the data is arranged this way, the workflow becomes much clearer. We embed each document, sentence, image caption, or product description ahead of time and store the resulting vector with the original record. Later, when a user submits a query, we embed the query into its own vector, compare it against the stored vectors, and return the records whose embeddings are closest. In other words, the useful structure is not just “a pile of vectors,” but a searchable collection of data where each vector remains tied to the object it represents.

This is the basic idea behind semantic retrieval. Embeddings let us search by meaning, not just by literal wording. That is powerful on its own, but real systems often combine semantic retrieval with lexical retrieval rather than choosing only one.

3 Hybrid Search

Once we understand lexical retrieval and semantic retrieval separately, the practical question becomes: how do we combine them? BM25 and semantic retrieval are not competing answers to the same problem. They are good at different things. BM25 is strong when exact wording matters. Vector retrieval is strong when meaning matters even though the words change.

3.1 Why Combine Lexical and Semantic Retrieval?

Neither retriever is universally better. If the query is “Toyota RAV4 AWD,” exact lexical matching should matter a lot. A result that contains those exact words is probably important. But if the query is “vehicle good for off-road adventures,” we still want to surface things like a Toyota RAV4, Ford F-150, or Subaru Outback even if the wording in the documents is different.

That is why hybrid retrieval is useful. We can run a lexical retriever such as BM25 and a semantic retriever based on embeddings, then combine their candidate lists. Score blending

is one option, but raw lexical scores and vector similarity scores often live on different scales. In practice, simple rank-based fusion methods are often easier to reason about.

3.2 Reciprocal Rank Fusion (RRF)

One common rank-based method is reciprocal rank fusion, usually shortened to RRF. The idea is simple: if a document appears near the top of several ranked lists, it should get a higher combined score.

$$\text{RRF}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

Here, d is a document or chunk. Each retriever contributes one term to the sum. $\text{rank}_i(d)$ is where that item appeared in retriever i 's ranked list. Smaller ranks contribute more, since being ranked 1 is better than being ranked 10. The constant k smooths the effect so that one top rank does not dominate too aggressively.

We can make that concrete with a small car-search example. Let us set $k = 10$ so the arithmetic stays easy to inspect. Suppose the user query is:

“Toyota RAV4 AWD for off-road adventures”

Now suppose BM25 and vector search each return a ranked list:

vehicle	BM25 rank	vector rank	RRF score	RRF rank
Toyota RAV4	1	4	$\frac{1}{11} + \frac{1}{14} \approx 0.162$	2
Ford F-150	2	1	$\frac{1}{12} + \frac{1}{11} \approx 0.174$	1
Subaru Outback	5	2	$\frac{1}{15} + \frac{1}{12} \approx 0.150$	3

BM25 strongly favors the Toyota RAV4 because the query contains those exact words. Vector search strongly favors the Ford F-150 because it looks semantically close to the “off-road adventures” part of the query. Ford wins the fused ranking because it is near the top of both lists. Toyota stays strong because rank 1 in BM25 still matters a lot. Subaru is rewarded for appearing near the top of both lists too, but in this example its combined score is still lower than Ford’s and Toyota’s. That is the appeal of RRF: it rewards agreement across retrievers without requiring their raw scores to live on the same scale.

3.3 From Fusion to Reranking

Hybrid retrieval gives us a better candidate set, but it still does not guarantee the final ordering is perfect. A common next step is to fuse the lexical and vector candidates first, then pass the merged list into a reranker that can compare the query and candidates more carefully.

At that point, we have a clear picture of the retrieval behavior we want: lexical search, semantic search, fusion, and then reranking. The next systems question is how to store that data and search it efficiently at scale.

4 From B-Trees to Vector Indexes

Before we talk about vector databases, it helps to look at how ordinary databases make search fast in the first place. The next couple of subsections cover database ideas that some readers may already know, so they can be skimmed if indexes and B-trees are familiar. They are still worth setting up because vector databases are built around the same basic split between storing data and searching it quickly.

4.1 Why Databases Need Indexes

The simplest way to search a table is to look through it row by row. If a database sees a query like this:

```
SELECT * FROM users WHERE id = 42;
```

it could check the first row, then the second row, then the third row, and keep going until it finds the matching record. This is a linear scan, often called a full table scan when the database has to consider the whole table.

That approach is simple and correct. For 100 rows, it may be fine. For 100 million rows, doing that on every request is a problem. The amount of work grows with the size of the table, even if the query only needs one record.

Indexes exist to avoid that kind of scan. A database index is a separate data structure built from selected column values and pointers back to the underlying rows. It does not replace the table. It gives the database a faster path to the rows that matter.

A scalar index is an index over one ordinary value per row, such as an integer ID, timestamp, username, price, or category. These are values that can be compared directly. Scalar indexes help with exact matches, ranges, and sorted order. Without an index, the database may have to scan each row and check whether `id = 42`. With an index on `id`, it can search the index first, find the row location, and then fetch the matching record.

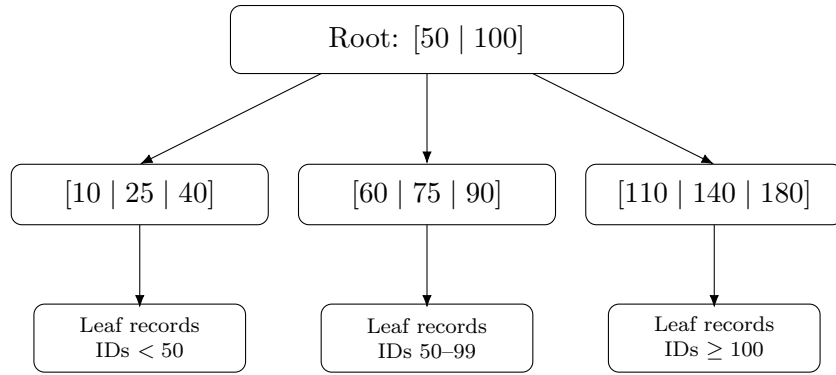
4.2 B-Trees: A Common Scalar Index

A B-tree is one common way to implement a scalar index. At a high level, a B-tree keeps keys in sorted order and stores them in a balanced tree. That balance matters because it keeps the tree from becoming a long lopsided chain. Instead of checking rows one by one, the database can follow the tree downward and quickly narrow the search to a smaller part of the data.

This is a good fit for the kinds of questions relational databases answer all the time:

```
SELECT * FROM users WHERE id = 42;  
SELECT * FROM orders WHERE created_at > '2026-01-01';
```

In both cases, the database is working with values that have a natural sorted order. An ID can be ordered numerically. A timestamp can be ordered chronologically. Once the keys are sorted, the database can ask a simple question: where would this value fall in that ordering?



The tree repeatedly narrows the search by following sorted key ranges.

Figure 3: A simple B-tree-style search intuition. The database does not scan every row. It follows ordered key ranges until it reaches the small part of the table that could contain the answer.

That is why B-trees are so useful. They are great when the thing being indexed has a clean one-dimensional ordering. Equality queries, range queries, and ordered scans all fit that pattern nicely.

4.3 Why This Breaks for Embeddings

Scalar indexes work because scalar values can be sorted in one clean order. Embeddings do not fit that pattern nearly as well. An embedding is not one number that can be dropped into a sorted list. It is a long vector such as

$$[0.12, -0.44, 0.91, \dots, 0.08].$$

Once the data looks like that, the query changes too. We are no longer asking:

```
WHERE id = 42
WHERE created_at > ...
```

Now we are asking something much stranger:

Which stored vectors are closest to this new vector under cosine similarity or distance?

That is a different kind of problem. A B-tree helps when the question is “where does this value belong in sorted order?” Vector search asks “which points are nearby in space?” There is no single sorted order of high-dimensional vectors that preserves semantic closeness in the way a sorted list of IDs preserves numeric order. Two vectors can be close in meaning even if many of their individual coordinates differ in messy ways. The whole geometry matters, not one field. That is why vector search needs a different kind of index.

This is the point where retrieval enters the story.

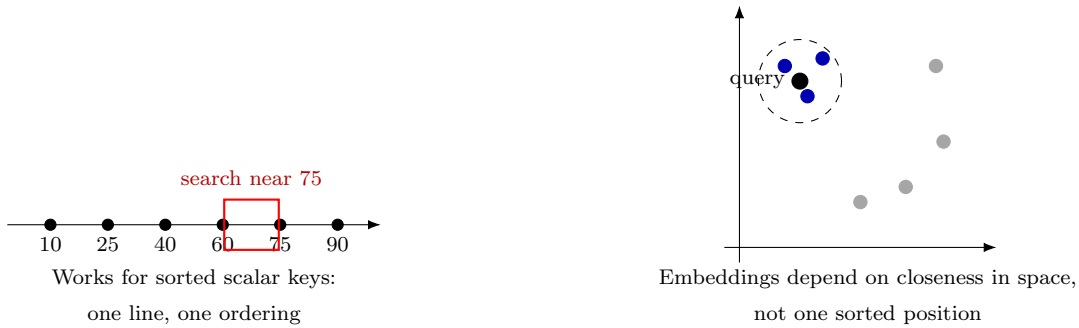


Figure 4: Why sorted order stops helping. On the left, a database can use a single ordered key to zoom in on the answer. On the right, semantic search depends on geometric closeness in a space of vectors.

4.4 What Retrieval Means Here

In the embedding setting, retrieval means this: given a query vector, return the stored records whose vectors are nearest to it under some similarity measure. Those results are called the query’s nearest neighbors.

That phrase can sound more mysterious than it really is. A nearest neighbor is just one of the stored vectors that sits closest to the query point in the embedding space. If we use cosine similarity, we are asking which vectors point in the most similar direction. If we use Euclidean distance, we are asking which vectors lie physically closest in that space.

4.5 Exact Search First

The simplest way to do this is also the most brute-force. We embed the query, compare it against every stored vector, compute a score for each one, sort the results, and return the top k . That is exact nearest-neighbor search.

It is easy to understand, and it gives the true top results for the chosen similarity measure. For a small dataset, that may be perfectly fine. But if the database holds millions of vectors, doing that much work for every query starts to hurt. The system may have to score the query against every row, even though only a handful of rows will make the final cut.

4.6 Approximate Search and HNSW

That scaling problem is why many vector databases use approximate nearest neighbor search, usually shortened to ANN. The word “approximate” is important. ANN methods do not promise the mathematically exact best answer every time. Instead, they try to return very good neighbors much faster than a full scan.

One of the most common ANN structures is HNSW, which stands for Hierarchical Navigable Small World. The name is a mouthful, but the basic idea is manageable. HNSW stores vectors in a graph. Each vector is linked to other vectors that are nearby. Search starts from a coarse upper layer, moves toward a promising region, and then drops into denser lower layers for a more local search. So instead of comparing the query with every stored vector,

the system walks through the graph toward better matches.

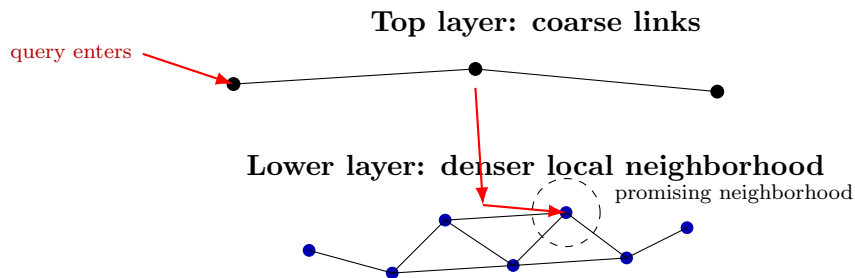


Figure 5: HNSW search intuition. The search starts with coarse navigation, then moves into a denser local neighborhood instead of scanning every vector in the database.

HNSW is not the only option. Another common ANN structure is IVFFlat, which groups vectors into coarse clusters first and then searches within the most relevant ones. Instead of walking a graph, IVFFlat asks a coarser question first: which regions of the vector space are worth looking inside?

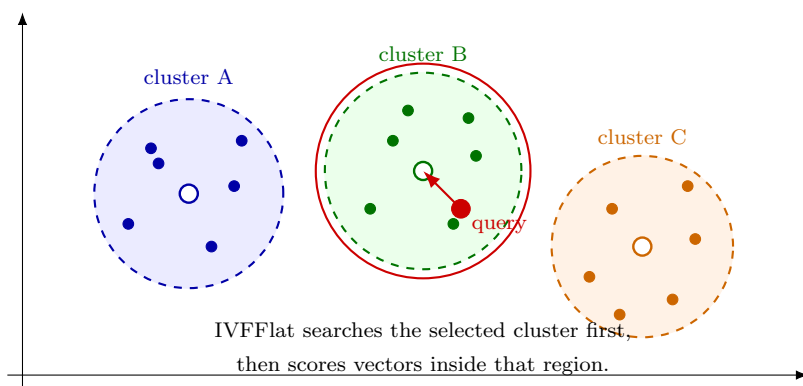


Figure 6: IVFFlat search intuition. The dashed circles are coarse regions, the larger filled dots are centroid vectors, and the smaller dots are stored vectors assigned to each centroid.

HNSW is still a good main example because it shows the key idea clearly: vector search usually depends on a special index designed for geometric similarity, not an ordinary sorted tree.

4.7 So What Does a Vector Database Store?

At this point, it helps to separate storage from indexing. A vector database still stores recognizable records. Each one usually includes the original content, an ID, the embedding, and whatever metadata the application needs.

```
{
  id: 42,
  content: "Toyota RAV4 is a compact SUV with good fuel economy",
```

```

embedding: [...],
metadata: {
  brand: "Toyota",
  type: "SUV",
  created_at: "2026-05-25"
}
}

```

Some of those fields may still use ordinary database indexes. For example, filtering by tenant, brand, timestamp, or category may rely on traditional indexes, often B-trees. That part of the system looks a lot like a normal database.

The difference is that the embeddings usually get an additional vector index built over them. That index is a separate data structure that points back to the stored records. In an HNSW-style index, that structure looks like a graph: each vector has links to nearby vectors, and the search walks through those links toward better matches.

In an IVFFlat-style index, the structure looks more like a set of buckets. The database stores a small set of centroid vectors, like the larger center dots in the IVFFlat figure above. Each centroid represents a coarse region of the vector space. Behind each centroid is an inverted list of row IDs or vector IDs assigned to that region:

```

centroid_1 -> [row_id 7, row_id 19, row_id 42, ...]
centroid_2 -> [row_id 3, row_id 14, row_id 88, ...]
centroid_3 -> [row_id 5, row_id 61, row_id 90, ...]

```

The original vectors and metadata still live in the records. The IVFFlat index just gives the database a faster way to find a smaller candidate set. At query time, the database compares the query vector to the centroids, chooses the closest few regions, and then scores the vectors in those lists. If it searches more lists, it is more likely to find the true nearest neighbors, but it also does more work.

That vector index is what makes nearest-neighbor retrieval fast enough to be practical. So a vector database is not just a place to store arrays of floats. It is a system that combines ordinary record storage with specialized similarity-search structures.

That is the distinction to keep in mind going forward. Ordinary database indexes answer questions about ordered keys. Vector indexes answer questions about geometric similarity in high-dimensional space.

Now that we know how retrieval should work and how vector search can be stored in practice, the next design question is what units of content should actually become vectors. In other words, what do we want to store, and how do we break it up intelligently so retrieval can work well?

5 What Should We Vectorize?

Once we understand how vectors can be stored and retrieved, the next question is more practical: what should become a vector in the first place?

The tempting answer is “the document.” If we have a PDF, embed the whole PDF. If we have a web page, embed the whole page. Sometimes that is fine, especially when the object is short and focused. But many real documents are too large or too mixed. A single document might contain an overview, installation instructions, troubleshooting notes, pricing details, and a legal disclaimer. One vector for the whole thing blurs all of those ideas together.

Instead, retrieval systems usually choose a unit of meaning. That unit might be a whole document, a paragraph, a section, a sentence, a product description, a support ticket, a code function, an image caption, or a slide. The right choice depends on what the user will ask and what kind of answer the system needs to return.

Large units preserve context. If we embed a full section, the retrieved result may include enough surrounding information to be useful on its own. The tradeoff is precision: the vector may represent several ideas at once, so a query can match the section for the wrong reason. Small units are more precise, but they can lose context. A sentence may match the query perfectly while depending on the previous paragraph to make sense.

So vectorization is not just a modeling step. It is a product decision about what the system is allowed to retrieve.

5.1 Chunking

Chunking is the process of splitting source material into retrievable pieces. It sounds mechanical, but it has a large effect on retrieval quality. A good embedding model cannot recover information that was cut into bad pieces.

The simplest strategy is fixed-size chunking. For example, split every document into chunks of 500 tokens, maybe with 50 tokens of overlap between neighboring chunks. This is easy to implement and works reasonably well when documents have consistent structure. The overlap is there because important ideas often cross chunk boundaries. Without it, a sentence at the end of one chunk and its explanation at the start of the next chunk may get separated.

Another strategy is structure-aware chunking. Instead of splitting every 500 tokens, we split on headings, paragraphs, list items, sections, code blocks, or other natural boundaries. This often produces chunks that feel more coherent because the chunk follows the shape of the document rather than an arbitrary token count.

Metadata matters too. A chunk should usually keep track of where it came from: document ID, title, section heading, page number, URL, timestamp, tenant, permissions, and any other fields needed at retrieval time. The metadata lets the system filter results, cite sources, and reconstruct context after retrieval.

The basic rule is simple: chunking defines what the retriever can find. If the chunks are too large, results may be vague. If they are too small, results may be contextless. If they are split in awkward places, the embedding may represent a fragment rather than a useful idea.

6 Quantization

Vectors are expensive to store at scale. A 1536-dimensional embedding stored as 32-bit floats uses 6144 bytes before any metadata or index overhead. A million of those vectors is already several gigabytes. A hundred million vectors changes the economics of the system.

Quantization is one way to reduce that cost. The idea is to store vectors with less precision than full 32-bit floating point values. Instead of keeping every coordinate as a full float, we store a compressed representation that is cheaper to keep in memory, cheaper to move through the CPU or GPU, and sometimes faster to search.

The tradeoff is distortion. Once we compress a vector, it is no longer exactly the same point in space. If the distortion is small, nearest neighbors mostly stay the same. If the distortion is large, the system may retrieve the wrong candidates.

There are several families of quantization:

- Scalar quantization stores each coordinate with fewer bits, such as 8-bit integers instead of 32-bit floats.
- Binary quantization compresses values very aggressively, often down to signs or bit patterns.
- Product quantization splits a vector into smaller subvectors and stores compact codes for each part.
- Newer methods try to preserve inner products and distances more carefully while still reducing memory.

The goal is not just smaller files. In retrieval, compression changes speed, memory use, cache behavior, and sometimes recall. A quantized index can be a huge win, but only if the search quality stays good enough for the application.

6.1 Quantization In Practice

Quantization is worth considering when vector storage or memory bandwidth becomes a bottleneck. That usually happens with large vector collections, high query volume, expensive memory, or systems that need to keep more of the index in RAM. It is less compelling when the dataset is small, the latency budget is loose, or retrieval quality is already fragile.

The right way to evaluate quantization is empirical. Compare the compressed system against a full-precision baseline on the same queries. Useful measurements include `recall@k`, `nDCG` or `MRR` if relevance labels exist, latency, index size, build time, memory usage, and end-to-end answer quality. If the retriever feeds a language model, the final question is not only whether the nearest neighbors changed. It is whether the generated answer still has the right evidence.

TurboQuant is one example of recent work in this area. The TurboQuant paper describes an online vector quantization method aimed at preserving geometric structure while reducing memory. The paper reports near-optimal distortion rates, inner-product preservation through a residual correction step, strong results for KV-cache compression, and improvements over product quantization baselines in nearest-neighbor search experiments.

Those are paper results, not a reason to blindly quantize every system. Whether a method like TurboQuant is useful depends on the embedding model, the dataset, the hardware, the index type, and the application’s tolerance for missed results. The practical test is straightforward: compress the vectors, run the same retrieval benchmark, and measure what changed. If storage and latency improve while recall and answer quality stay within the target range, quantization is doing its job. If the system starts missing important evidence, the memory savings are not worth it.